

Sigint example 1:

```
affan@affan-VirtualBox: ~/Desktop/lab8
affan@affan-VirtualBox:~/Desktop/lab8$ cat activity1.c
#include<stdio.h>
#include<unistd.h>
#include<wait.h>
#include<signal.h>
int state=0;
void signal_handler(int num){
    printf("\nI am here because ctrl+c was pressed\n");
}
int main()
{
    signal(SIGINT,signal_handler);
    while(1){
        sleep(1);
        printf("process running\n");
        state=1;
    }
    return 0;
}
affan@affan-VirtualBox:~/Desktop/lab8$ gcc activity1.c -o a1
affan@affan-VirtualBox:~/Desktop/lab8$ ./a1
process running
process running
process running
^C
I am here because ctrl+c was pressed
process running
^Z
[2]+  Stopped                  ./a1
affan@affan-VirtualBox:~/Desktop/lab8$
```

Sigint example 2:

```
affan@affan-VirtualBox:~/Desktop/lab8$ gcc activity2.c -o a2
affan@affan-VirtualBox:~/Desktop/lab8$ cat activity2.c
#include<stdio.h>
#include<unistd.h>
#include<wait.h>
#include<signal.h>
int state=0;
void signal_handler(int num){
    if(state==0){
        printf("process paused\n");
        state=1;
    }
    if(state==1){
        printf("process resumed\n");
        state=0;
    }
    signal(SIGINT,SIG_DFL);
}
int main()
{
    signal(SIGINT,signal_handler);
    while(1){
        sleep(1);
        printf("process running\n");
    }
    return 0;
}
affan@affan-VirtualBox:~/Desktop/lab8$ ./a2
process running
process running
process running
process running
process running
process running
^Cprocess paused
process resumed
process running
process running
process running
process running
process running
process running
^C
affan@affan-VirtualBox:~/Desktop/lab8$
```

Sigquit example:

```
affan@affan-VirtualBox: ~/Desktop/lab8
affan@affan-VirtualBox:~/Desktop/lab8$ gcc activity3.c -o a3
affan@affan-VirtualBox:~/Desktop/lab8$ cat activity3.c
#include<stdio.h>
#include<unistd.h>
#include<wait.h>
#include<signal.h>

void signal_handler(int SIGNO){
    if(SIGNO==SIGINT){
        printf("\nreceived SIGINT\n");
    }
    else if(SIGNO==SIGQUIT){
        printf("\nreceived SIGQUIT\n");
    }
}

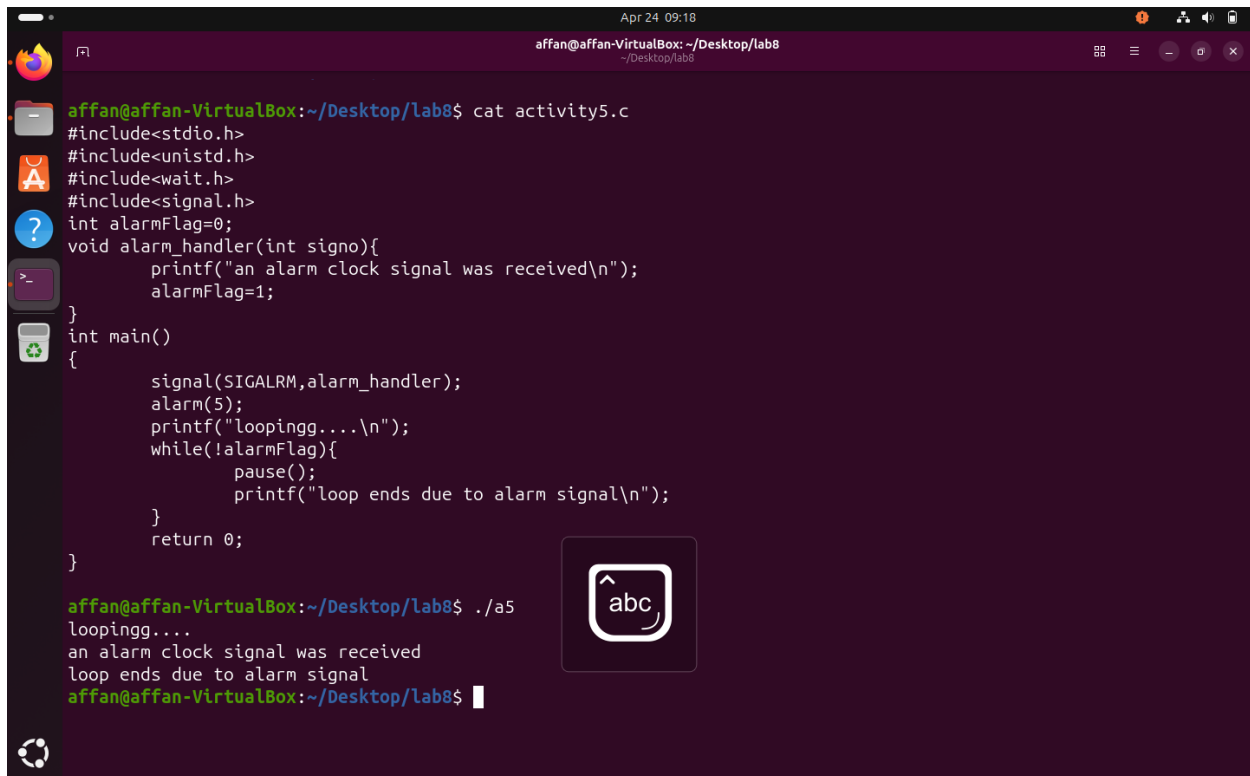
int main()
{
    if(signal(SIGINT,signal_handler)==SIG_ERR){
        printf("\ncan't catch SIGINT\n");
    }
    if(signal(SIGQUIT,signal_handler)==SIG_ERR){
        printf("\ncan't catch SIGQUIT\n");
    }
    while(1){
        sleep(1);
        printf("process running\n");
    }
    return 0;
}

affan@affan-VirtualBox:~/Desktop/lab8$ ./a3
process running
process running
process running
^C
received SIGINT
process running
process running
process running
^C
received SIGQUIT
process running
process running
process running
^C
received SIGQUIT
process running
process running
process running
^C
received SIGINT
process running
process running
process running
^Z
[4]+  Stopped                  ./a3
affan@affan-VirtualBox:~/Desktop/lab8$
```

Sigtstp example:

```
affan@affan-VirtualBox: ~/Desktop/lab8
affan@affan-VirtualBox:~/Desktop/lab8$ gcc activity4.c -o a4
affan@affan-VirtualBox:~/Desktop/lab8$ ./a4
process running
process running
process running
^Z - cannot execute ctrl+z
process running
process running
^Z - cannot execute ctrl+z
process running
process running
^Z - cannot execute ctrl+z
process running
process running
^C
affan@affan-VirtualBox:~/Desktop/lab8$
```

Alarm signal example:

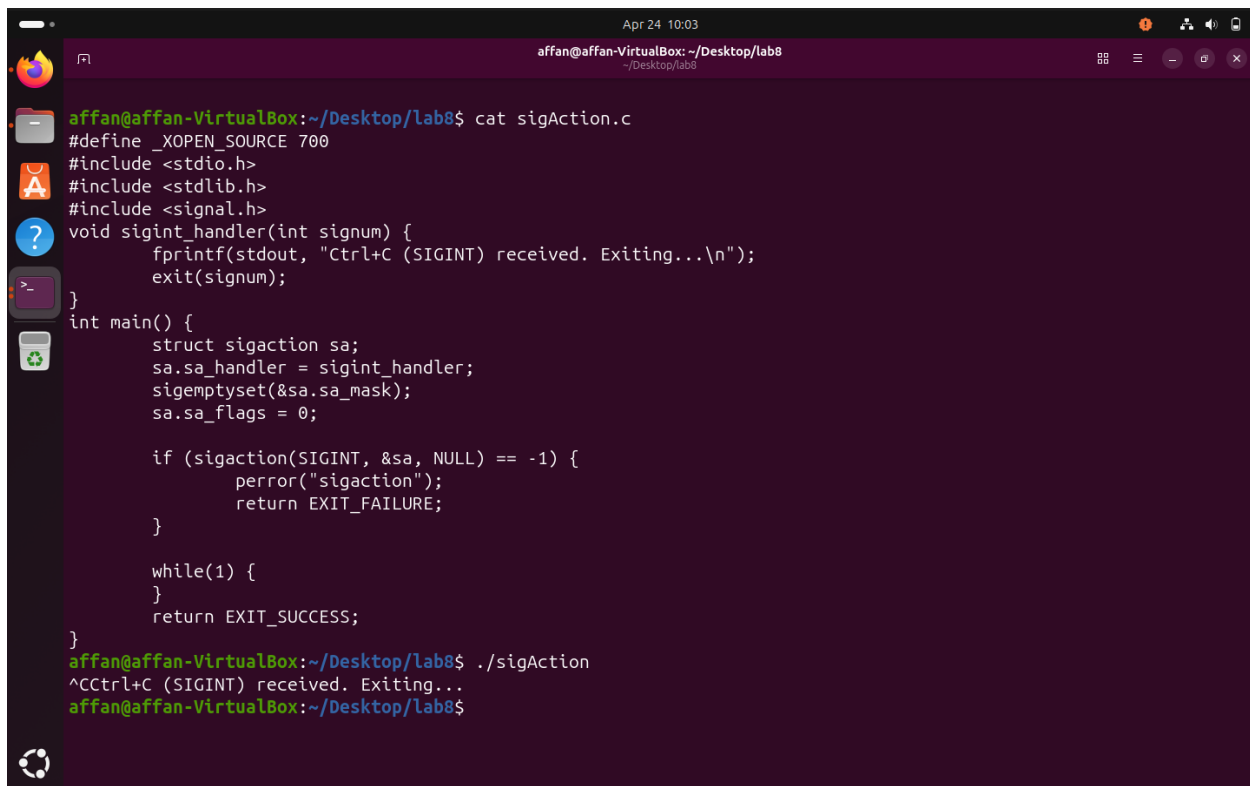


A terminal window titled "affan@affan-VirtualBox: ~/Desktop/lab8" showing the execution of a C program named activity5.c. The program includes headers for stdio, unistd, wait, and signal. It defines an alarmFlag variable and an alarm_handler function that prints a message and sets the flag. The main function sets the alarm to 5 seconds, prints "loopingg...\\n", and enters a while loop that pauses and prints "loop ends due to alarm signal\\n" until the alarm flag is set. The program is compiled and run with ./a5, resulting in the expected output.

```
affan@affan-VirtualBox:~/Desktop/lab8$ cat activity5.c
#include<stdio.h>
#include<unistd.h>
#include<wait.h>
#include<signal.h>
int alarmFlag=0;
void alarm_handler(int signo){
    printf("an alarm clock signal was received\\n");
    alarmFlag=1;
}
int main()
{
    signal(SIGALRM,alarm_handler);
    alarm(5);
    printf("loopingg...\\n");
    while(!alarmFlag){
        pause();
        printf("loop ends due to alarm signal\\n");
    }
    return 0;
}

affan@affan-VirtualBox:~/Desktop/lab8$ ./a5
loopingg....
an alarm clock signal was received
loop ends due to alarm signal
affan@affan-VirtualBox:~/Desktop/lab8$
```

sigAction example:



A terminal window titled "affan@affan-VirtualBox: ~/Desktop/lab8" showing the execution of a C program named sigAction.c. The program defines a _XOPEN_SOURCE 700 macro and includes headers for stdio, stdlib, and signal. It defines a sigint_handler function that prints a message and exits with the signal number. The main function sets up a sigaction structure with the handler, sets the mask to empty, and sets flags to 0. It then calls sigaction(SIGINT, &sa, NULL) and checks for failure. If successful, it enters a while(1) loop. The program is compiled and run with ./sigAction, resulting in the expected output when Ctrl+C is pressed.

```
affan@affan-VirtualBox:~/Desktop/lab8$ cat sigAction.c
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
void sigint_handler(int signum) {
    fprintf(stdout, "Ctrl+C (SIGINT) received. Exiting...\\n");
    exit(signum);
}
int main() {
    struct sigaction sa;
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return EXIT_FAILURE;
    }

    while(1) {
    }
    return EXIT_SUCCESS;
}

affan@affan-VirtualBox:~/Desktop/lab8$ ./sigAction
^C Ctrl+C (SIGINT) received. Exiting...
affan@affan-VirtualBox:~/Desktop/lab8$
```

Multithreaded example:

```
affan@affan-VirtualBox: ~/Desktop/lab8
affan@affan-VirtualBox:~/Desktop/lab8$ gcc multiThreaded.c -o multiThreaded
affan@affan-VirtualBox:~/Desktop/lab8$ ./multiThreaded
Main Thread ID: 126969470900032, Parent Process ID: 3665, Process ID: 6438
Thread 126969470900032 received SIGUSR1 signal (Parent Process ID: 3665, Process ID: 6438)
Thread 126969450583744 received SIGUSR1 signal (Parent Process ID: 3665, Process ID: 6438)
^C
affan@affan-VirtualBox:~/Desktop/lab8$ cat multiThreaded.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <signal.h>
#include <unistd.h>
#define NUM_THREADS 4
pthread_t threads[NUM_THREADS];
void sigusr1_handler(int signum) {
    pthread_t tid = pthread_self();
    pid_t pid = getpid();
    pid_t ppid = getppid();
    printf("Thread %lu received SIGUSR1 signal (Parent Process ID: %u, Process ID: %u)\n", tid, ppid, pid);
}
void *thread_function(void *arg) {
    signal(SIGUSR1, sigusr1_handler);
    while (1) {
        sleep(1);
    }
    return NULL;
}
int main() {
    for (int i = 0; i < NUM_THREADS; ++i) {
        if (pthread_create(&threads[i], NULL, thread_function, NULL) != 0) {
            perror("pthread_create");
            exit(EXIT_FAILURE);
        }
    }
    pthread_t main_tid = pthread_self();
    pid_t pid = getpid();
    pid_t ppid = getppid();
    printf("Main Thread ID: %lu, Parent Process ID: %u, Process ID: %u\n", main_tid, ppid, pid);
    kill(pid, SIGUSR1);
    pthread_kill(threads[2], SIGUSR1);
    for (int i = 0; i < NUM_THREADS; ++i) {
        if (pthread_join(threads[i], NULL) != 0) {
            perror("pthread_join");
            exit(EXIT_FAILURE);
        }
    }
    return 0;
}
```

sigFork example:

```
affan@affan-VirtualBox: ~/Desktop/lab8
affan@affan-VirtualBox:~/Desktop/lab8$ gcc sigFork.c -o sigFork
affan@affan-VirtualBox:~/Desktop/lab8$ cat sigFork.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <time.h>

void proc_exit(int sig) {
    int wstat;
    pid_t pid;
    while (1) {
        pid = wait3(&wstat, WNOHANG, NULL);
        if (pid == 0 || pid == -1) {
            fprintf(stdout, "return value of wait3() is %d\n", pid);
            return;
        }
        fprintf(stdout, "Return code: %d\n", wstat);
    }
}

int main() {
    srand(time(NULL));
    signal(SIGCHLD, proc_exit);
    switch (fork()) {
        case -1:
            perror("fork");
            exit(EXIT_FAILURE);
        case 0:
            printf("Child process (PID: %d) is alive temporarily\n", getpid());
            int ret_code = rand() % 100;
            printf("Child process exiting with return code: %d\n", ret_code);
            exit(ret_code);
        default:
            printf("Parent process (PID: %d) waiting for child termination...\n", getpid());
            pause();
    }
    return 0;
}
```

```
affan@affan-VirtualBox:~/Desktop/lab8$ ./sigFork
Parent process (PID: 6622) waiting for child termination...
Child process (PID: 6623) is alive temporarily
Child process exiting with return code: 26
Return code: 6656
return value of wait3() is -1
affan@affan-VirtualBox:~/Desktop/lab8$
```